Befehlsreferenz PC → Test-PCB (für das Python-Programm)

Diplomarbeit „Analyse von Degradationseffekten bei Feldeffekttransistoren“ · Firmware PCB_test_code

Stand 08.11.2026

1 Zweck

Diese Doku beschreibt, **welche Befehle das Python-Programm in welcher Reihenfolge schicken muss**, damit die Firmware `PCB_test_code` eine bestimmte Messung ausführt: ID-VG, Transient, Stress (BTI/HCD, statisch/dynamisch), Stress mit Nachmessung, Temperatur-Wartezeit und MSM (Measure–Stress–Measure).

Grundlage ist der aktuelle Code in `COMMAND_DECODE.c`, `SCPI_COMMANDS.h` und `main.c`. Die Messroutinen in `measure.c` (`IDVG()`, `transient()`, `stress()`, `wait_temp_equi()`, `MSM()`) sind **noch leer**. Das hier beschriebene Verhalten ist deshalb das *vorgesehene* Verhalten; was die Firmware zurückschickt, ist noch nicht festgelegt (siehe Kapitel 9).

2 Verbindung und Übertragungsformat

Punkt	Wert
Schnittstelle	UART1 über USB-UART (isoliert)
Baudrate	**9600** (aktuell in `main.c`: `uart1_set_baud(9600)`), 8N1. Höhere Baudrate (z. B. 115200) ist geplant
Startmeldung	Nach Reset schickt die Firmware `FETCH READY\r\n`. Erst danach Befehle senden
Befehlsformat	`NAME= wert`: **genau ein Leerzeichen nach dem =**, kein Leerzeichen davor
Zeilenende	`\n` (oder `\r`). Ein `\r\n` schadet nicht (die zweite, leere Zeile wird ignoriert)
max. Länge	127 Zeichen pro Zeile (`UART_RX_BUFFER_SIZE = 128`)
Groß-/Kleinschreibung	**wird unterschieden**, z. B. `SOURce1_Sweep_TIme=` mit großem `I`, `Temp_equilibrium_wait=` klein
Zahlen	Dezimalpunkt (`0.1`, nicht `0,1`). Zeiten in **ms** als ganze Zahl
Bestätigung	**Keine.** Die Firmware antwortet auf Parameter-Befehle nicht
Unbekannte Befehle	werden **stillschweigend ignoriert**. Tippfehler fallen also nicht auf

2.1 Wichtig: Pause zwischen den Befehlen

Der UART-Interrupt nimmt nur dann neue Zeichen an, wenn die vorige Zeile schon von der Hauptschleife abgeholt wurde (`uart_rx_complete == false`). Schickt das Python-Programm mehrere Zeilen direkt hintereinander, **können Zeichen bzw. ganze Befehle verloren gehen**, vor allem wenn die Hauptschleife gerade eine Messfunktion ausführt.

Empfehlung für das Python-Programm:

- nach jeder Zeile **ca. 20 ms warten** (bei 9600 Baud dauert ein Zeichen schon ≈ 1 ms);
- **Parameter zuerst, Start-Befehl zuletzt** schicken;
- während einer laufenden Messung nur ABORT schicken.

2.2 Python-Grundgerüst (pyserial)

```
import time
import serial

ser = serial.Serial("COM5", 9600, timeout=1)   # COM-Port anpassen

def warte_auf_ready(timeout=5):
    t0 = time.time()
    while time.time() - t0 < timeout:
        zeile = ser.readline().decode(errors="ignore").strip()
        if zeile == "FETCH READY":
            return True
    return False

def send(name, wert):
    zeile = f"{name}= {wert}\n"          # genau ein Leerzeichen nach '='
    ser.write(zeile.encode("ascii"))
    time.sleep(0.02)                     # Firmware Zeit zum Abholen geben

def send_list(befehle):
    for name, wert in befehle:
        send(name, wert)

def abort():
    ser.write(b"ABORT\n")
```

3 Alle Befehle (Parameter)

SOURce1 = **Gate**, SOURce2 = **Drain** (DAC A = Gate, DAC B = Drain; laut eigener Doku, am Schaltplan prüfen).

3.1 SOURce (Spannungsquellen)

Befehl	Wert	Bedeutung
SOURce1_VOLTage= / SOURce2_VOLTage=	Zahl [V]	fixe Spannung Gate / Drain (Transient, Lesespannung bei MSM)
SOURce<n>_Sweep_Start=	Zahl [V]	Sweep-Startwert
SOURce<n>_Sweep_Stop=	Zahl [V]	Sweep-Endwert
SOURce<n>_Sweep_Step=	Zahl [V]	Schrittweite

Befehl	Wert	Bedeutung
SOURce<n>_Sweep_TIme=	Zahl [ms]	Einschwingzeit pro Schritt (Standard-Empfehlung 5 ms)
SOURce<n>_PROT=	Zahl [V]	Spannungsgrenze
SOURce<n>_RANGE=	25 / 50	Ausgangsbereich, **als Zahl schicken**
SOURce<n>_OUTput=	ON / OFF	Ausgang ein/aus
SOURce<n>_AUTOrange=	ON / OFF	Autorange

3.2 SENSE (Strommessung)

Befehl	Wert	Bedeutung
SENSE_CURRENT=	Zahl	Stromziel
SENSE_RANGE=	0 / 1	0 = 100 nA (Rf 10 MΩ), 1 = 10 µA (Rf 100 kΩ), **als Zahl schicken**
SENSE_AUTORANGE=	ON / OFF	Autorange Strombereich
SENSE_MODE_GND=	ON / OFF	TIA-Eingang auf GND (Schutz beim Stressen)
SENSE_MODE_AZERO=	ON / OFF	Auto-Zero
SENSE_MODE_ZCORRECT_AQUIRE=	Zahl	Offset-Korrekturwert
SENSE_MODE_AVERAGE=	ganze Zahl	Anzahl Mittelungen pro Messpunkt (Standard 100)
SENSE_MODE_MEASUREMENT=	START / STOP	wird decodiert, aber in `main.c` derzeit **nicht ausgewertet**
SENSE_MODE_IDVG=	TRUE / FALSE	**startet** eine ID-VG-Messung
SENSE_MODE_TRANSIENT=	TRUE / FALSE	**startet** eine Transient-Messung

3.3 STRESS

Befehl	Wert	Bedeutung
STRESS_TYPE=	BTI / HCD	BTI: nur Gate, Drain = 0 V · HCD: Gate + Drain
STRESS_MODE=	STATIC / DYNAMIC	Dauerstress / gepulst
STRESS_VOLTage=	Zahl [V]	Gate-Stressspannung
STRESS_DRAIN_VOLTage=	Zahl [V]	Drain-Stressspannung (nur HCD)
STRESS_TIME=	ganze Zahl [ms]	Gesamtdauer (STATIC)
STRESS_ON_TIME=	ganze Zahl [ms]	Stress pro Puls (DYNAMIC)
STRESS_OFF_TIME=	ganze Zahl [ms]	Pause pro Puls (DYNAMIC)
STRESS_CYCLES=	ganze Zahl	Anzahl Pulse (DYNAMIC)
STRESS_AFTER_IDVG=	TRUE / FALSE	nach dem Stress ID-VG messen
STRESS_AFTER_TRANSIENT=	TRUE / FALSE	nach dem Stress Transient messen
STRESS_OUTput=	ON / OFF	**startet** den Stress

3.4 TEMP

Befehl	Wert	Bedeutung
`Temp_equilibrium_wait=`	`TRUE` / `FALSE`	**startet** das Warten auf thermisches Gleichgewicht

3.5 MSM (Measure–Stress–Measure)

Befehl	Wert	Standard	Bedeutung
`MSM_CYCLES=`	ganze Zahl	5	Anzahl Stress→Mess-Zyklen
`MSM_START_TIME=`	ganze Zahl [ms]	1000	erste Stressdauer
`MSM_TIME_FACTOR=`	Zahl	10	Stresszeit pro Zyklus × Faktor (1 s, 10 s, 100 s …)
`MSM_MEAS_TYPE=`	`SPOT` / `IDVG` / `TRANSIENT`	SPOT	was in der Messphase gemessen wird
`MSM_MEAS_DELAY=`	ganze Zahl [ms]	1	Stress aus → erster Messwert
`MSM_RECOVERY_STRESS_TIME=`	`TRUE` / `FALSE`	TRUE	TRUE: Recovery-Zeit = Stresszeit des Zyklus
`MSM_RECOVERY_TIME=`	ganze Zahl [ms]	0	feste Recovery-Zeit, **nur** bei `MSM_RECOVERY_STRESS_TIME= FALSE`
`MSM_INITIAL_MEASURE=`	`TRUE` / `FALSE`	TRUE	Referenzmessung vor dem ersten Stress
`MSM_MAX_POINTS=`	ganze Zahl	5000	max. Messpunkte pro Messung (Buffergröße)
`SENSE_MODE_MSM=`	`TRUE` / `FALSE`	FALSE	**startet** die MSM-Messung

Recovery-Zeit wird **nur bei `TRANSIENT`** verwendet. Bei `SPOT` und `IDVG` startet der nächste Stress direkt nach der Messung.

3.6 ABORT

`ABORT` (oder `ABOR`) bricht eine laufende Messung, einen Stress oder die Temperatur-Wartezeit sofort ab. Er wird direkt im UART-Interrupt erkannt und funktioniert auch während einer Messung.

4 Start-Befehle und Reihenfolge in der Hauptschleife

Fünf Befehle **starten** etwas. Die Hauptschleife prüft sie nach jedem empfangenen Befehl in dieser Reihenfolge:

Reihenfolge	Start-Befehl	ruft auf
1	`SENSE_MODE_IDVG= TRUE`	`IDVG()`
2	`SENSE_MODE_TRANSIENT= TRUE`	`transient()`
3	`STRESS_OUTput= ON`	`stress()`
4	`Temp_equilibrium_wait= TRUE`	`wait_temp_equi()`
5	`SENSE_MODE_MSM= TRUE`	`MSM()`

Regeln für das Python-Programm:

- **Immer nur einen Start-Befehl pro Messung** schicken, und zwar als **letzte** Zeile.
- Die Temperatur-Wartezeit kommt in der Schleife *nach* IDVG/Transient/Stress. Soll vorher gewartet werden, zuerst `Temp_equilibrium_wait= TRUE` allein schicken, das Ende abwarten und erst dann die Messung starten.

- Die Firmware setzt die Start-Flags derzeit **nicht zurück** (offener Punkt in `measure.c`). Bis das umgesetzt ist, nach der Messung das passende `...= FALSE` bzw. `STRESS_OUTput= OFF` schicken.
- Alle Parameter bleiben gespeichert, bis sie überschrieben werden oder die Platine neu startet. Das Python-Programm sollte deshalb **vor jeder Messung alle relevanten Parameter neu schicken** und die nicht benötigten Start-Flags explizit auf FALSE/OFF setzen.

5 Gemeinsamer Grund-Block

Dieser Block kommt vor jeder Messart:

```
SENSE_RANGE= 0                  # 0 = 100 nA, 1 = 10 µA
SENSE_AUTORANGE= OFF
SENSE_MODE_AVERAGE= 100
SENSE_MODE_GND= OFF             # TIA-Eingang ans DUT (beim Start liegt er auf GND)
SENSE_MODE_IDVG= FALSE          # alle Start-Flags sicher aus
SENSE_MODE_TRANSIENT= FALSE
STRESS_OUTput= OFF
Temp_equilibrium_wait= FALSE
SENSE_MODE_MSM= FALSE
```

In den folgenden Beispielen steht dafür `[Grund-Block]`. Die Zahlen sind Beispielwerte.

6 Einzelmessungen

6.1 ID-VG

Drain fix, Gate wird von Start bis Stop in Schritten gefahren.

```
[Grund-Block]
SOURce2_VOLTage= 0.1            # Drain fix
SOURce1_Sweep_Start= 0
SOURce1_Sweep_Stop= 2.5
SOURce1_Sweep_Step= 0.05
SOURce1_Sweep_TIme= 5           # Einschwingzeit pro Schritt [ms]
SENSE_MODE_IDVG= TRUE           # Start
```

6.2 Transient

Gate und Drain fix, Strom wird über die Zeit aufgezeichnet.

```
[Grund-Block]
SOURce1_VOLTage= 1.5            # Gate fix
SOURce2_VOLTage= 0.1            # Drain fix
SENSE_MODE_TRANSIENT= TRUE      # Start
```

Hinweis: Für Dauer und Abtastrate des Transienten gibt es noch keinen eigenen Befehl.

7 Stress

7.1 BTI statisch

```
[Grund-Block]
STRESS_TYPE= BTI
STRESS_MODE= STATIC
STRESS_VOLTage= 5.0
STRESS_DRAIN_VOLTage= 0         # bei BTI ohnehin 0 V
```

```
STRESS_TIME= 60000              # 60 s
STRESS_AFTER_IDVG= FALSE
STRESS_AFTER_TRANSIENT= FALSE
STRESS_OUTput= ON               # Start
```

7.2 BTI dynamisch

```
[Grund-Block]
STRESS_TYPE= BTI
STRESS_MODE= DYNAMIC
STRESS_VOLTage= 5.0
STRESS_DRAIN_VOLTage= 0
STRESS_ON_TIME= 100             # 100 ms Stress
STRESS_OFF_TIME= 100            # 100 ms Pause
STRESS_CYCLES= 600
STRESS_AFTER_IDVG= FALSE
STRESS_AFTER_TRANSIENT= FALSE
STRESS_OUTput= ON
```

7.3 HCD statisch

```
[Grund-Block]
STRESS_TYPE= HCD
STRESS_MODE= STATIC
STRESS_VOLTage= 5.0
STRESS_DRAIN_VOLTage= 3.0
STRESS_TIME= 60000
STRESS_AFTER_IDVG= FALSE
STRESS_AFTER_TRANSIENT= FALSE
STRESS_OUTput= ON
```

7.4 HCD dynamisch

Wie 7.2, aber mit `STRESS_TYPE= HCD` und einer `STRESS_DRAIN_VOLTage` größer als 0.

Achtung (Stressing.md): Hohe Drain-Spannung bei leitendem Kanal kann zu hohem Drain-Strom führen und TIA bzw. Drain-Treiber beschädigen. Bei HCD Strombereich und Spannungsgrenzen (`SOURce<n>_PROT=`) prüfen.

7.5 Stress mit anschließender Messung

Hier gelten die Parameter der Messung (6.1 oder 6.2) **und** des Stresses. Gestartet wird nur mit `STRESS_OUTput= ON`, **nicht** zusätzlich mit `SENSE_MODE_IDVG= TRUE`.

Stress → ID-VG:

```
[Grund-Block]
SOURce2_VOLTage= 0.1
SOURce1_Sweep_Start= 0
SOURce1_Sweep_Stop= 2.5
SOURce1_Sweep_Step= 0.05
SOURce1_Sweep_TIme= 5
STRESS_TYPE= BTI
STRESS_MODE= STATIC
STRESS_VOLTage= 5.0
STRESS_TIME= 60000
STRESS_AFTER_IDVG= TRUE
```

```
STRESS_AFTER_TRANSIENT= FALSE
STRESS_OUTput= ON
```

Stress → Transient: wie oben, aber SOURce1_VOLTage= und SOURce2_VOLTage= statt der Sweep-Werte sowie STRESS_AFTER_IDVG= FALSE und STRESS_AFTER_TRANSIENT= TRUE.

Wenn **beide** STRESS_AFTER_* TRUE sind, ist die Reihenfolge der zwei Messungen nicht festgelegt. Sie hängt davon ab, wie `stress()` geschrieben wird.

7.6 Vorher/Nachher (ΔV_{th})

1. ID-VG wie 6.1 (Referenz), Ende abwarten, SENSE_MODE_IDVG= FALSE.
2. Stress mit STRESS_AFTER_IDVG= TRUE wie 7.5.

8 MSM (Measure–Stress–Measure)

Für MSM gilt:

- Die **Stressspannungen** kommen aus STRESS_TYPE, STRESS_VOLTage und STRESS_DRAIN_VOLTage.
- STRESS_MODE= STATIC schicken. STRESS_TIME, ON_TIME, OFF_TIME und CYCLES werden nicht verwendet, die Zeiten kommen aus MSM_*.
- Die **Lesespannungen** kommen aus SOURce1_VOLTage (Gate) und SOURce2_VOLTage (Drain), bei IDVG aus den Sweep-Parametern.
- **Nicht** STRESS_OUTput= ON, SENSE_MODE_IDVG= TRUE oder SENSE_MODE_TRANSIENT= TRUE schicken. Sonst läuft zusätzlich eine eigene Messung. Die Messart wählt MSM_MEAS_TYPE.
- Start nur mit SENSE_MODE_MSM= TRUE.

Gemeinsamer MSM-Block (in den Beispielen [MSM-Block]):

```
[Grund-Block]
STRESS_TYPE= BTI                # oder HCD
STRESS_MODE= STATIC
STRESS_VOLTage= 5.0
STRESS_DRAIN_VOLTage= 0         # bei HCD > 0
MSM_CYCLES= 5
MSM_START_TIME= 1000            # 1 s, 10 s, 100 s, 1000 s, 10000 s
MSM_TIME_FACTOR= 10
MSM_MEAS_DELAY= 1
MSM_INITIAL_MEASURE= TRUE
MSM_MAX_POINTS= 5000
```

8.1 MSM mit SPOT

Ein Messpunkt pro Zyklus bei t = MSM_MEAS_DELAY. Danach geht es **sofort** mit dem nächsten Stress weiter; die Recovery-Zeit wird nicht abgewartet.

```
[MSM-Block]
SOURce1_VOLTage= 1.5            # Lese-Gatespannung
SOURce2_VOLTage= 0.1            # Lese-Drainspannung
MSM_MEAS_TYPE= SPOT
SENSE_MODE_MSM= TRUE            # Start
```

8.2 MSM mit IDVG

Ein ID-VG-Sweep pro Zyklus. Die Dauer ergibt sich aus den Sweep-Parametern; die Recovery-Zeit wird nicht verwendet.

```
[MSM-Block]
SOURce2_VOLTage= 0.1
SOURce1_Sweep_Start= 0
SOURce1_Sweep_Stop= 2.5
SOURce1_Sweep_Step= 0.05
SOURce1_Sweep_TIme= 5
MSM_MEAS_TYPE= IDVG
SENSE_MODE_MSM= TRUE
```

Tipp: Der Sweep unterbricht den Stress. Für wenig Recovery einen kurzen Sweep wählen (wenige Punkte um V_{th}).

8.3 MSM mit TRANSIENT, Recovery-Zeit = Stresszeit

`MSM_MAX_POINTS` Punkte, logarithmisch verteilt von `MSM_MEAS_DELAY` bis zur Stresszeit des jeweiligen Zyklus.

```
[MSM-Block]
SOURce1_VOLTage= 1.5
SOURce2_VOLTage= 0.1
MSM_MEAS_TYPE= TRANSIENT
MSM_RECOVERY_STRESS_TIME= TRUE
SENSE_MODE_MSM= TRUE
```

8.4 MSM mit TRANSIENT, feste Recovery-Zeit

```
[MSM-Block]
SOURce1_VOLTage= 1.5
SOURce2_VOLTage= 0.1
MSM_MEAS_TYPE= TRANSIENT
MSM_RECOVERY_STRESS_TIME= FALSE
MSM_RECOVERY_TIME= 1000        # 1 s, muss > 0 sein
SENSE_MODE_MSM= TRUE
```

8.5 Übersicht: welche Parameter bei welcher MSM-Variante gebraucht werden

TR-S = TRANSIENT mit Recovery = Stresszeit · TR-F = TRANSIENT mit fester Recovery-Zeit

Parameter	SPOT	IDVG	TR-S	TR-F
STRESS_TYPE	✓	✓	✓	✓
STRESS_VOLTage	✓	✓	✓	✓
STRESS_DRAIN_VOLTage	✓	✓	✓	✓
MSM_CYCLES	✓	✓	✓	✓
MSM_START_TIME	✓	✓	✓	✓
MSM_TIME_FACTOR	✓	✓	✓	✓
MSM_MEAS_DELAY	✓	✓	✓	✓
MSM_INITIAL_MEASURE	✓	✓	✓	✓
SOURce1_VOLTage (Gate)	✓		✓	✓
SOURce2_VOLTage (Drain)	✓	✓	✓	✓
SOURce1_Sweep_*		✓		
MSM_MAX_POINTS			✓	✓
MSM_RECOVERY_STRESS_TIME			TRUE	FALSE
MSM_RECOVERY_TIME				> 0

8.6 Datenmenge und Übertragung

- Gesendet wird **binär**, mit 2 Byte pro Punkt.
- 5000 Punkte sind 10 kB. Das dauert bei 115200 Baud ≈ 0,87 s, bei 9600 Baud ≈ 10,4 s.
- Die Daten einer Messung werden während des **nächsten** Stresses geschickt; nach dem letzten Zyklus am Ende.
- Daraus folgt: `MSM_START_TIME` muss größer sein als die Übertragungszeit plus Reserve. Bei 115200 Baud ist 1 s knapp, dann lieber `MSM_MAX_POINTS= 4000`. Bei 9600 Baud ist MSM mit 5000 Punkten erst ab etwa 15 s Stress sinnvoll.

9 Offene Punkte (betreffen das Python-Programm)

- **Rückgabeformat** der Messdaten (Header, Zeitstempel, binär/ASCII, Ende-Kennung) ist noch nicht festgelegt. Ebenso fehlt eine Meldung „Messung fertig“.
- **Keine Bestätigung** von Befehlen. Eine OK/ERR-Antwort im Decoder würde das Python-Programm robuster machen.
- **Start-Flags** werden von der Firmware noch nicht zurückgesetzt (siehe Kapitel 4).
- **Baudrate** ist noch 9600. Für MSM und Transient wird 115200 oder mehr gebraucht; das Limit setzen USB-UART-Chip und Optokoppler.
- Die Defines in `SCPI_COMMANDS.h` für `SOURce_RANGE`, `SENSE_RANGE`, `STRESS_TYPE` und `STRESS_MODE` erzeugen mit Enum-Namen einen Text, den der Decoder nicht versteht (z. B. `STRESS_BTI` statt `BTI`). Das Python-Programm muss daher **die Werte aus dieser Doku** schicken.
- Für `SENSE_MODE_ZCORRECT=` gibt es keinen Decode-Zweig. Abfragen mit ? werden ebenfalls noch nicht beantwortet.
- Für den Transienten (6.2) gibt es noch keine Befehle für Dauer und Abtastrate.

10 Quellen

- Eigener Code: `Software/firmware/PCB_test_code/COMMAND_DECODE.c`, `COMMAND_DECODE.h`, `SCPI_COMMANDS.h`, `main.c`, `UART.c`, `UART.h`
- Eigene Doku: `Dokumentation/Firmware_Doku_PCB_test_code.pdf`
- Eigene Doku: `Dokumentation/Messroutinen_IDVG_Transient.pdf`
- Eigene Doku: `Dokumentation/Stressing.md`
- pySerial-Dokumentation: https://pyserial.readthedocs.io/en/latest/
- SCPI-99 Standard: https://www.ivifoundation.org/downloads/SCPI/scpi-99.pdf
- STMicroelectronics RM0008 (USART): `Software/firmware/Datenblätter_Cortex/`
- B. Kaczer et al., „Ubiquitous Relaxation in BTI Stressing – New Evaluation and Insights“, IEEE IRPS 2008 (MSM-Ablauf)
- T. Grasser et al., „The Universality of NBTI Relaxation and its Implications for Modeling and Characterization“, IEEE IRPS 2007 (Recovery = Stresszeit)